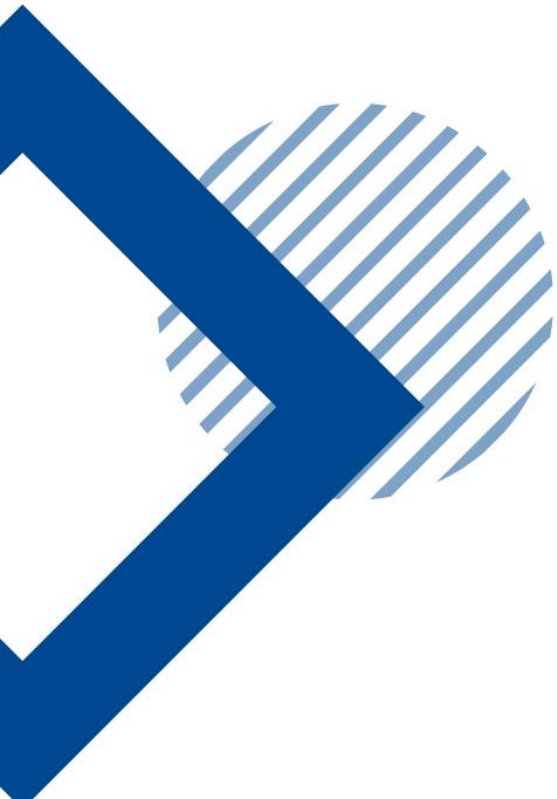
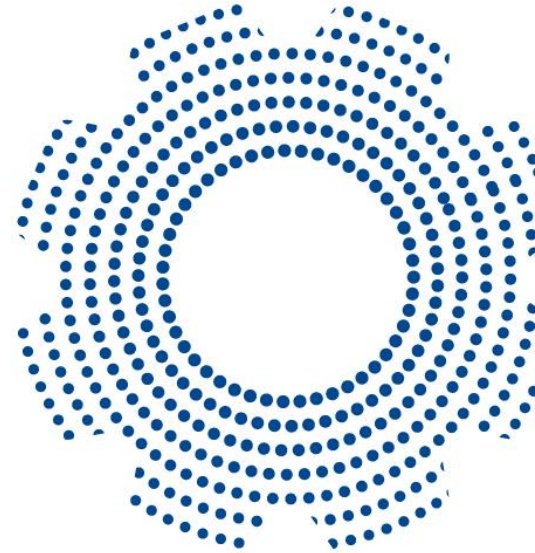
Инверсия управления, внедрение зависимостей

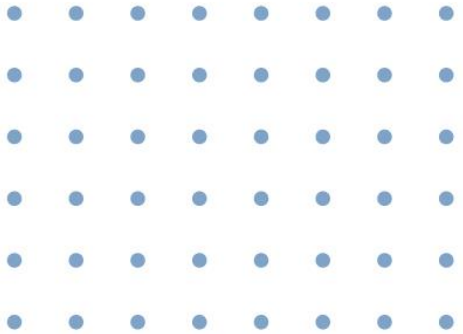
Интернет программирование











2022

Инверсия управления – 1

Инверсия управления (Inversion of Control, IoC) – принцип в ООП, позволяющий уменьшить степень связанности (сцепления) кода.

При использовании IoC программа представляет собой множество зависимых компонентов.

Поток управления программной системы формируется при этом не программистом, а фреймворком, реализующим принципы IoC.

Фреймворк определяет каким образом связать компоненты и создает экземпляры необходимых классов.

Инверсия управления – 2

Преимущества IoC:

- Уменьшение степени зависимости между классами.
- Упрощение программного кода.
- Упрощение тестирования программной системы.
- Повышение гибкости и скорости разработки программной системы.

Недостатки IoC:

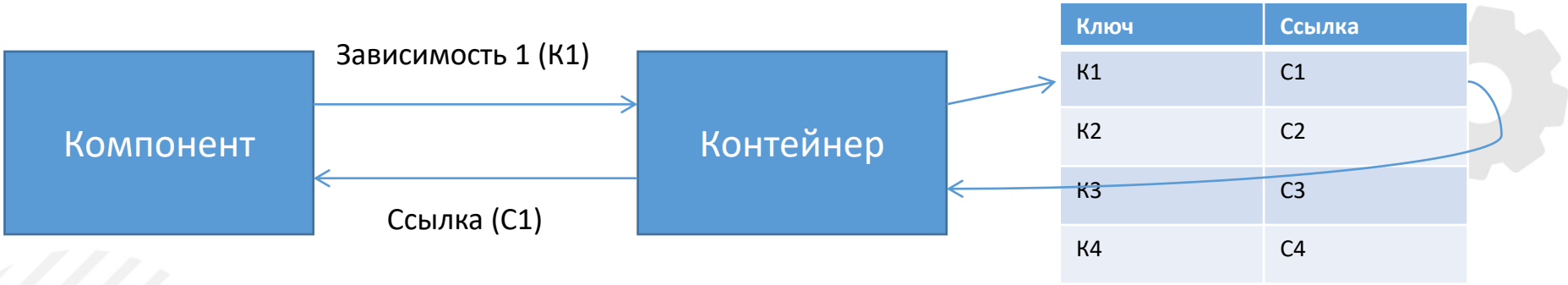
- Зависимость от фреймворка.
- Логика программной системы разбросана по отдельным файлам.
- Неочевидный поток управления.
- Магия.

Инверсия управления – 3

Типы IoC:

1. Поиск зависимостей (Dependency Lookup, DL) – получение ссылки на зависимость самим компонентом:
2. Внедрение зависимостей (Dependency Injection, DI) – внедрение зависимости контейнером IoC.

Инверсия управления – 4



Ключ	Ссылка
К1	С1
К2	С2
К3	С3
К4	С4

Поиск зависимостей

Типы DL:

1. Извлечение зависимостей (Dependency Pull). Поиск компонента в реестре контейнера осуществляется по идентификатору.

```
public class MyClass {
   public void do() {
      Container c = new Container(…);
      SomeClass d = c.getDependency(“name”, SomeClass.class);
   }
}
```

2. Контекстуализированный поиск зависимостей (Contextualized Dependency Lookup). Аналогично извлечению зависимостей, но работа с контейнером осуществляется через реализацию компонентом соответствующего интерфейса. Контейнер сам вызывает необходимый метод интерфейса при готовности.

```
interface Component {
   void setDependency(Container c);
}
public class MyClass implements Component {
   private SomeClass d;
   public void setDependency(Container c) {
      d = c.getDependency(“name”, SomeClass.class);
   }
}
```

Внедрение зависимостей

Типы DI:

1. Внедрение через конструктор (Constructor Dependency Injection). Используется в большинстве случаев при написании программных систем, так как гарантирует наличие всех зависимостей при создании объекта класса.

```
public class MyClass {
    private final SomeClass d;
    public MyClass(SomeClass d) {
        this.d = d;
    }
}
```

2. Внедрение через set-метод (Setter Dependency Injection). Используется в случае необходимости гибкой конфигурации параметров объекта. Некоторые параметры могут быть не заданы.

```
public class MyClass {
    private SomeClass d;
    public setD(SomeClass d) {
        this.d = d;
    }
}
```

Внедрение или поиск зависимостей?

В большинстве случаев предпочтительно использовать внедрение зависимостей.

Поиск зависимостей используется:
1. Если контейнер не поддерживает внедрение.
2. Если задача требует поиск зависимостей.

Преимущества внедрения зависимостей:
1. Классы не привязаны к конкретному контейнеру.
2. Классы не содержат в себе «лишние» импорты и операторы.
3. Простой код → меньше ошибок.

IoC в Spring

Фреймворк Spring основан на принципе IoC и поддерживает:

- Поиск зависимостей.
- Внедрение зависимостей.

IoC в Spring реализован на основе принципа поиска компонентов (component scanning).

Для определения компонентов используются специальные аннотации.

При запуске приложения на основе Spring фреймворк ищет компоненты, формирует дерево зависимостей и пытается их разрешить.

После успешного запуска программы контейнер содержит ссылки на все найденные компоненты.

Поиск компонентов в Spring

Настройка процесса поиска компонентов производится через аргументы аннотации @ComponentScan

```
@ComponentScan(basePackages = “ru…”)
```

Аннотация @ComponentScan без аргументов ищет компоненты в текущем пакете (package) и всех его подпакетах.

Несколько аннотаций @ComponentScan можно объединить с помощью аннотации @ComponentScans

```
@ComponentScans({
   @ComponentScan(basePackages = “ru…”)
   @ComponentScan(basePackages = “en…”)
})
```

В Spring Boot обычно не требуется настройка поиска компонентов.

Аннотацию @ComponentScan необходимо использовать совместно с аннотацией @Configuration

Аннотации компонентов в Spring

Следующие аннотации фреймворка Spring учитываются при поиске компонентов:

- @Bean – аннотирует функцию, возвращающую ссылку на объект-компонент.
- @Component (базовая) – аннотирует класс-компонент.
- @Repository – аннотирует компонент для работы с источниками данных.
- @Service – аннотирует класс-компонент с бизнес-логикой.
- @Controller – аннотирует контроллеры.
- @Configuration – аннотирует компоненты для конфигурации.

Название экземпляра компонента формируется из названия класса, первая буква которого представлена в нижнем регистре.

Название компонента можно переопределить через аргументы аннотации.

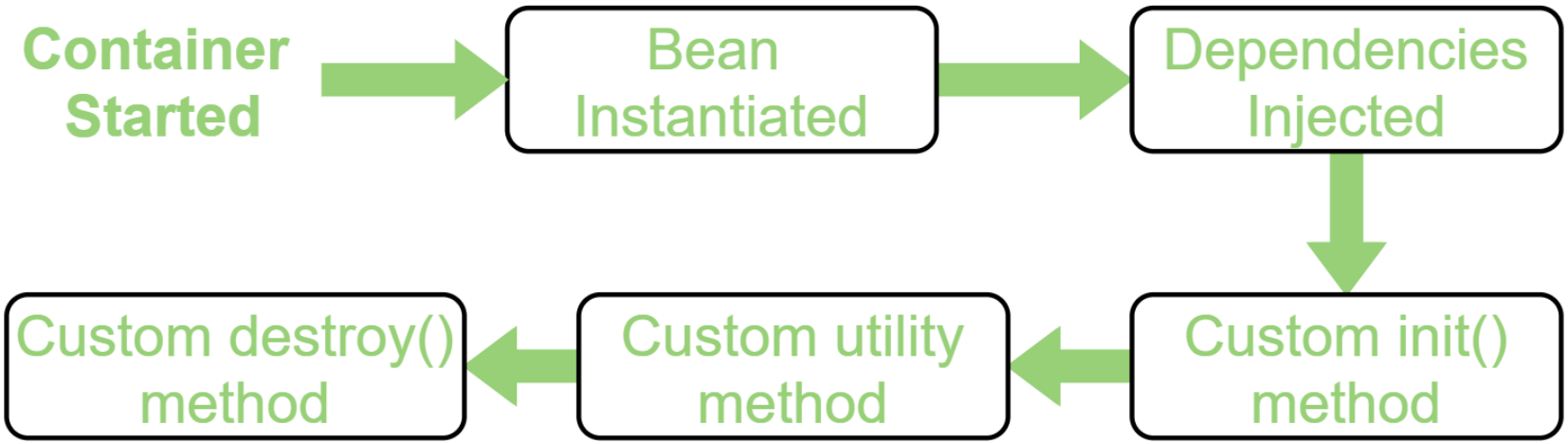
Области видимости компонентов в Spring

Следующие области видимости доступны для компонентов, полученных с помощью аннотации @Bean:

- singleton (по-умолчанию) – компонент создается в одном экземпляре, любые изменения в объекте будут видны по всем ссылкам на него.
- prototype – каждый раз контейнер возвращает ссылку на новый объект.
- request – контейнер возвращает ссылку на новый объект в рамках каждого HTTP-запроса.
- session – контейнер возвращает ссылку на новый объект в рамках каждой HTTP-сессии.
- application – аналогично singleton, но с учетом различных контекстов сервлетов.
- websocket – контейнер возвращает ссылку на новый объект в рамках каждой Websocket-сессии.

Области видимости request, session, application и websocket доступны только в Web-приложениях.

Жизненный цикл компонентов в Spring



Пример – 1

```
public interface Speaker {
   String say();
}

public class SpeakerDeu implements Speaker {
   @Override
   public String say() {
      return "Hallo";
   }
}

public class SpeakerRus implements Speaker {
   @Override
   public String say() {
      return "Привет";
   }
}

public class SpeakerEng implements Speaker {
   @Override
   public String say() {
      return "Hello";
   }
}
```

Пример – 2

```
@SpringBootApplication
@RestController
public class SbappApplication {
   private final Logger log = LoggerFactory.getLogger(SbappApplication.class);
   private static ApplicationContext context;

   @Bean("ru")
   public Speaker createRusSpeaker() {
      log.info("Call createRusSpeaker()");
      return new SpeakerRus();
   }

   @Bean("en")
   public Speaker createEngSpeaker() {
      log.info("Call createEngSpeaker()");
      return new SpeakerEng();
   }

   @Bean("de")
   public Speaker createDeuSpeaker() {
      log.info("Call createDeuSpeaker()");
      return new SpeakerDeu();
   }

   public static void main(String[] args) {
       context = SpringApplication.run(SbappApplication.class, args);
   }

   @GetMapping("/")
   public String hello(@RequestParam(value = "name", defaultValue = "Мир") String name,
                       @RequestParam(value = "lang", defaultValue = "ru") String lang) {
       final Speaker speaker = (Speaker) context.getBean(lang);
       return String.format("%s %s!", speaker.say(), name);
   }
}
```

Пример – 3

```
@Configuration
public class SpeakerConfiguration {
   private final Logger log = LoggerFactory.getLogger(SpeakerConfiguration .class);
   @Bean("ru")
   public Speaker createRusSpeaker() {
      …
   }
   @Bean("en")
   public Speaker createEngSpeaker() {
      …
   }
   @Bean("de")
   public Speaker createDeuSpeaker() {
      …
   }
}

@SpringBootApplication
@RestController
public class SbappApplication {
private static ApplicationContext context;

   public static void main(String[] args) {
      context = SpringApplication.run(SbappApplication.class, args);
   }

   @GetMapping("/")
   public String hello(@RequestParam(value = "name", defaultValue = "Мир") String name,
                       @RequestParam(value = "lang", defaultValue = "ru") String lang) {
      final Speaker speaker = (Speaker) context.getBean(lang);
      return String.format("%s %s!", speaker.say(), name);
   }
}
```

Пример – 4

```
@Component(value = "de")
public class SpeakerDeu implements Speaker {
   …
}

@Component(value = "ru")
public class SpeakerRus implements Speaker {
   …
}

@Component(value = "en")
public class SpeakerEng implements Speaker {
   …
}

@SpringBootApplication
@RestController
public class SbappApplication {
private static ApplicationContext context;

   public static void main(String[] args) {
      context = SpringApplication.run(SbappApplication.class, args);
   }

   @GetMapping("/")
   public String hello(@RequestParam(value = "name", defaultValue = "Мир") String name,
                       @RequestParam(value = "lang", defaultValue = "ru") String lang) {
      final Speaker speaker = (Speaker) context.getBean(lang);
      return String.format("%s %s!", speaker.say(), name);
   }
}
```

Пример – 5

```
@RestController
public class SpeakerController {
   private final ApplicationContext context;

   public SpeakerController(ApplicationContext context) {
      this. context = context;
   }

   @GetMapping("/")
   public String hello(@RequestParam(value = "name", defaultValue = "Мир") String name,
                       @RequestParam(value = "lang", defaultValue = "ru") String lang) {
      final Speaker speaker = (Speaker) context.getBean(lang);
      return String.format("%s %s!", speaker.say(), name);
   }
}

@SpringBootApplication
public class SbappApplication {
   public static void main(String[] args) {
      SpringApplication.run(SbappApplication.class, args);
   }
}
```

Пример – 6

```
@Service
public class SpeakerService {
   private final ApplicationContext applicationContext;

   public SpeakerService(ApplicationContext applicationContext) {
      this.applicationContext = applicationContext;
   }

    public String say(String name, String lang) {
       final Speaker speaker = (Speaker) applicationContext.getBean(lang);
       return String.format("%s %s!", speaker.say(), name);
    }
}

@RestController
public class SpeakerController {
   private final SpeakerService speakerService;

   public SpeakerController(SpeakerService speakerService) {
      this.speakerService = speakerService;
   }

   @GetMapping("/")
   public String hello(@RequestParam(value = "name", defaultValue = "Мир") String name,
                       @RequestParam(value = "lang", defaultValue = "ru") String lang) {
      return speakerService.say(name, lang);
   }
}

@SpringBootApplication
public class SbappApplication {
   public static void main(String[] args) {
      SpringApplication.run(SbappApplication.class, args);
   }
}
```

Пример – 7

```
@Bean(value = "ru", initMethod = "init", destroyMethod = "destroy")
public SpeakerRus createRusSpeaker() {
   …
}

public class SpeakerRus implements Speaker {
   private final Logger log = LoggerFactory.getLogger(SpeakerRus.class);

   @Override
   public String say() {
      return "Привет";
   }
   public void init() {
      log.info("SpeakerRus.init()");
   }
   public void destroy() {
      log.info("SpeakerRus.destroy()");
   }
}
```

Пример – 8

```
@Bean("en")
public Speaker createEngSpeaker() {
   …
}

@Component(value = "en")
public class SpeakerEng implements Speaker {
   …
}
```

```
public class SpeakerEng implements Speaker, InitializingBean, DisposableBean {
   private final Logger log = LoggerFactory.getLogger(SpeakerEng.class);
   @Override
   public String say() {
      return "Hello";
   }
   @Override
   public void afterPropertiesSet() {
      log.info("SpeakerEng.afterPropertiesSet()");
   }
   @Override
   public void destroy() {
      log.info("SpeakerEng.destroy()");
   }
}
```

Пример – 9

```
@Bean(“de")
public Speaker createDeuSpeaker() {
   …
}

@Component(value = “de")
public class SpeakerDeu implements Speaker {
   …
}

public class SpeakerDeu implements Speaker {
   private final Logger log = LoggerFactory.getLogger(SpeakerDeu.class);
   @Override
   public String say() {
      return "Hallo";
   }
   @PostConstruct
   public void init() {
      log.info("SpeakerDeu.init()");
   }
   @PreDestroy
   public void destroy() {
      log.info("SpeakerDeu.destroy()");
   }
}
```

Пример – 10

```
@SpringBootTest
class SbappApplicationTests {
    @Autowired
    SpeakerService speakerService;

    @Test
    void testSpeakerRus() {
        final String res = speakerService.say("Мир", "ru");
        Assertions.assertEquals("Привет Мир!", res);
    }
    @Test
    void testSpeakerEng() {
        final String res = speakerService.say("World", "en");
        Assertions.assertEquals("Hello World!", res);
    }
    @Test
    void testSpeakerDeu() {
        final String res = speakerService.say("Welt", "de");
        Assertions.assertEquals("Hallo Welt!", res);
    }
    @Test
    void testSpeakerErrorWired() {
        Assertions.assertThrows(NoSuchBeanDefinitionException.class, () -> speakerService.say("Мир", "rus"));
    }
}
```

Спасибо за внимание!

Литература:

1. Изучаем Java EE 7. – СПб.: Питер, 2014. – 640 с.
2. Spring 4 для профессионалов, 4-е изд. : Пер. с англ. – М. : ООО «И.Д. Вильямс», 2015. – 752 с.
3. Spring Documentation, Core Technologies. URL: https://docs.spring.io/spring-framework/docs/current/reference/html/core.html



2022